

Day 17: Clean Phone Number

1. Day Number

Day 17

2. Topic Name

`replace()` and basic string cleaning

3. Connection

Earlier, you cleaned an account holder's name using methods such as `strip()` and `title()`.

Today, you will clean a phone number entered with unwanted spaces or dashes.

For example:

```
98765-43210
```

will become:

```
9876543210
```

4. Important Topics

- `replace()`
- `strip()`
- Strings
- Method chaining
- Basic string validation idea

5. Foundational Notes

What is string cleaning?

String cleaning means removing unwanted characters or spaces from text before using it.

A user might enter the same phone number in different formats:

```
98765 43210
98765-43210
98765-43210
```

Your program should convert them into one consistent format:

```
9876543210
```

The `replace()` method

`replace()` replaces one part of a string with another.

Syntax:

```
new_text = old_text.replace("old_value", "new_value")
```

To remove something, replace it with an empty string:

```
text.replace("-", "")
```

Example:

```
phone = "123-456"  
clean_phone = phone.replace("-", "")
```

Result:

```
123456
```

The `strip()` method

`strip()` removes spaces from the beginning and end of a string.

```
phone = " 123456 "  
clean_phone = phone.strip()
```

Result:

```
123456
```

It does not remove spaces from the middle:

```
123 456
```

To remove middle spaces, use:

```
phone.replace(" ", "")
```

Strings do not change automatically

String methods return a new string. Therefore, save the returned value:

```
clean_text = text.replace("-", "")
```

Method chaining

You can apply multiple string methods in one statement:

```
clean_text = text.strip().replace("-", "")
```

The methods run from left to right.

6. Easy Example

This example removes spaces and dashes from a product code:

```
product_code = "  AB-12 34  "

clean_code = product_code.strip()
clean_code = clean_code.replace(" ", "")
clean_code = clean_code.replace("-", "")

print(clean_code)
```

Output:

```
AB1234
```

This is only an example of string cleaning, not the full phone-number solution.

7. Problem Statement

Create a Python program that:

1. Asks the user to enter a phone number.
2. The phone number may contain spaces or dashes.
3. Removes spaces from the phone number.
4. Removes dashes from the phone number.
5. Prints the cleaned phone number.

Example:

```
Enter phone number: 98765-43210
Clean phone number: 9876543210
```

8. Concepts Used

input()

Used to receive the phone number from the user.

```
phone = input("Enter phone number: ")
```

`input()` returns a string, which is suitable because a phone number is normally treated as text.

strip()

Removes unnecessary spaces before and after the phone number.

replace()

Removes spaces and dashes by replacing them with empty strings.

Function

The cleaning logic can be placed inside a reusable function.

Return value

The function can return the cleaned phone number to the main program.

String validation idea

After cleaning, you can think about checking:

- Is the string empty?
- Does it contain only digits?
- Does it have the expected number of digits?

For today's main problem, focus primarily on removing spaces and dashes.

9. Thought Process

Step 1: Receive the phone number

Ask the user to enter a phone number.

Do not convert it to `int`, because:

- Phone numbers are not used for calculations.
- Some phone numbers may begin with 0.
- The input may initially contain spaces or dashes.

Step 2: Remove outside spaces

Use `strip()` to remove spaces before and after the input.

Step 3: Remove spaces inside the number

Use `replace(" ", "")`.

This changes:

```
98765 43210
```

into:

```
9876543210
```

Step 4: Remove dashes

Use `replace("-", "")`.

This changes:

```
98765-43210
```

into:

```
9876543210
```

Step 5: Check for empty input

After cleaning, check whether anything remains.

If the cleaned phone number is empty, print an appropriate message.

Step 6: Return the result

The function should return the cleaned value to the main program.

Step 7: Print the clean phone number

The main program receives the returned value and displays it.

10. Pseudocode

```
START

DEFINE a function that accepts phone number

    Remove spaces from beginning and end

    Remove all spaces from phone number

    Remove all dashes from phone number

    RETURN cleaned phone number

ASK user to enter phone number

CALL the cleaning function

IF cleaned phone number is empty
    PRINT empty phone number message
ELSE
    PRINT cleaned phone number

END
```

11. Suggested Solving Approach: Functional Approach

Create one function responsible for cleaning the phone number.

Possible function structure:

```
def clean_phone_number(phone):
    # Clean the phone number
    # Return the cleaned value
```

Main program structure:

```
# Get input

# Call the function

# Check the returned result

# Print the result
```

This approach is useful because the same function can clean many phone numbers later.

For example:

```
first_cleaned_phone = clean_phone_number(first_phone)
second_cleaned_phone = clean_phone_number(second_phone)
```

12. Easy Edge Cases

Phone number with spaces

Input:

98765 43210

Expected cleaned value:

9876543210

Phone number with dashes

Input:

98765-43210

Expected cleaned value:

9876543210

Phone number with both spaces and dashes

Input:

98765 - 43210

Expected cleaned value:

9876543210

Empty phone number

Input:

Possible output:

Phone number cannot be empty.

Only spaces and dashes

Input:

- -

After cleaning, the result will be empty. Treat it like an empty phone number.

13. Expected Input

Example 1:

98765-43210

Example 2:

98765 43210

Example 3:

98765 - 43210

14. Expected Output

For input:

98765-43210

Output:

Clean phone number: 9876543210

For input:

98765 43210

Output:

Clean phone number: 9876543210

For empty input:

Phone number cannot be empty.

15. Hint Only

Inside your function:

1. Use `strip()` first.
2. Replace " " with "".
3. Replace "- " with "".
4. Return the cleaned string.
5. In the main program, check whether the returned string is empty before printing it.

Useful pattern:

```
cleaned_value = original_value.replace(character_to_remove, "")
```

Remember: do not convert the phone number to an integer. Keep it as a string.